Resumo do Código Implementado

Este notebook implementa um sistema de autocompletar usando a estrutura de dados Árvore Trie, conforme solicitado no enunciado do Trabalho Prático 3. O dicionário usado para montar a árvore é composto pelas palavras reservadas da linguagem Python, obtidas por `keyword.kwlist`, simulando o comportamento de uma IDE que sugere comandos ao programador durante a digitação.

A implementação lê o dicionário, insere cada palavra na Trie e permite buscar palavras a partir do início digitado pelo usuário. A saída principal do programa é uma lista, exibida em tela, contendo todas as palavras do dicionário que começam com o prefixo informado. Para representar a leitura letra a letra pedida no enunciado, a execução interativa mostra as sugestões para cada prefixo parcial da entrada digitada.

Além da implementação dos algoritmos, o notebook inclui uma rotina para exibir visualmente a árvore montada em HTML/SVG.

Estrutura simplificada do notebook

- Importações e definição dos recursos usados.
- Definição da estrutura do nó da Trie.
- Leitura do dicionário de palavras reservadas de Python.
- Implementação da inserção de palavras na Trie.
- Construção da árvore Trie a partir do dicionário.
- Implementação da busca por palavra exata.
- Implementação da busca por prefixo.
- Implementação do autocompletar.
- Execução letra a letra das sugestões para o usuário.
- Execução interativa para o usuário digitar o início de uma palavra.
- Rotina para exibir a árvore montada.
- Funções auxiliares para visualização didática das buscas.
- Visualização didática da busca exata, da busca por prefixo e do autocompletar.
- Resultados finais pedidos no enunciado.

Lista das `keyword.kwlist` (35 palavras)

Coluna 1	Coluna 2	Coluna 3	Coluna 4	Coluna 5
False	await	else	import	pass
None	break	except	in	raise
True	class	finally	is	return
and	continue	for	lambda	try
as	def	from	nonlocal	while
assert	del	global	not	with
async	elif	if	or	yield

In [1]:

```python
# Descrição: esta célula importa os recursos necessários para montar a Trie,
# carregar o dicionário de palavras reservadas de Python e exibir a árvore
# e as execuções didáticas em HTML/SVG dentro do notebook.

from dataclasses import dataclass, field
import html
import keyword
from IPython.display import HTML, display
```

In [2]:

```python
# Descrição: esta célula define a estrutura básica de um nó da Árvore Trie.
# Cada nó armazena seus filhos em um dicionário, informa se encerra uma palavra
# completa e guarda a palavra completa quando o nó é terminal.

@dataclass
class NoTrie:
    filhos: dict[str, "NoTrie"] = field(default_factory=dict)
    fim_de_palavra: bool = False
    palavra: str | None = None
```

In [3]:

```python
# Descrição: esta célula lê o dicionário usado para montar a árvore.
# O enunciado permite usar palavras reservadas de uma linguagem de programação;
# aqui são usadas as palavras reservadas oficiais da linguagem Python.

DICIONARIO_PYTHON = sorted(keyword.kwlist)

print(f"Quantidade de palavras no dicionário: {len(DICIONARIO_PYTHON)}")
print(DICIONARIO_PYTHON)
```

```
Quantidade de palavras no dicionário: 35
['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break', 'class', 'continue', 'def', 'del', 'elif', 'else', 'except', 'finally', 'for', 'from', 'glob
al', 'if', 'import', 'in', 'is', 'lambda', 'nonlocal', 'not', 'or', 'pass', 'raise', 'return', 'try', 'while', 'with', 'yield']
```

In [4]:

```python
# Descrição: esta célula implementa a inserção de uma palavra na Trie.
# Para cada caractere da palavra, o algoritmo desce um nível na árvore,
# criando o nó quando ele ainda não existe.
# Complexidade: O(m) no tempo, em que m é o tamanho da palavra inserida;
# O(m) no espaço no pior caso, quando nenhum caractere do caminho ainda existe.

def inserir_palavra(raiz, palavra):
    no_atual = raiz

    for caractere in palavra:
        if caractere not in no_atual.filhos:
            no_atual.filhos[caractere] = NoTrie()
        no_atual = no_atual.filhos[caractere]

    no_atual.fim_de_palavra = True
    no_atual.palavra = palavra
```

In [5]:

```python
# Descrição: esta célula monta a Árvore Trie completa a partir do dicionário.
# Cada palavra reservada de Python é inserida individualmente na estrutura.
# Complexidade: O(T) no tempo, em que T é a soma dos tamanhos de todas as palavras;
# O(T) no espaço no pior caso, quando há pouco compartilhamento de prefixos.

def montar_trie(palavras):
    raiz = NoTrie()

    for palavra in palavras:
        inserir_palavra(raiz, palavra)

    return raiz

raiz_trie = montar_trie(DICIONARIO_PYTHON)
raiz_trie
```

Out[5]:

```
NoTrie(filhos={'F': NoTrie(filhos={'a': NoTrie(filhos={'l': NoTrie(filhos={'s': NoTrie(filhos={'e': NoTrie(filhos={}, fim_de_palavra=True, palavra='False')}, fim_de_pa
lavra=False, palavra=None)}, fim_de_palavra=False, palavra=None)}, fim_de_palavra=False, palavra=None)}, fim_de_palavra=False, palavra=None), 'N': NoTrie(filhos={'o':
NoTrie(filhos={'n': NoTrie(filhos={'e': NoTrie(filhos={}, fim_de_palavra=True, palavra='None')}, fim_de_palavra=False, palavra=None)}, fim_de_palavra=False, palavra=No
ne)}, fim_de_palavra=False, palavra=None), 'T': NoTrie(filhos={'r': NoTrie(filhos={'u': NoTrie(filhos={'e': NoTrie(filhos={}, fim_de_palavra=True, palavra='True')}, fi
m_de_palavra=False, palavra=None)}, fim_de_palavra=False, palavra=None)}, fim_de_palavra=False, palavra=None), 'a': NoTrie(filhos={'n': NoTrie(filhos={'d': NoTrie(filh
os={}, fim_de_palavra=True, palavra='and')}, fim_de_palavra=False, palavra=None), 's': NoTrie(filhos={'s': NoTrie(filhos={'e': NoTrie(filhos={'r': NoTrie(filhos={'t':
NoTrie(filhos={}, fim_de_palavra=True, palavra='assert')}, fim_de_palavra=False, palavra=None)}, fim_de_palavra=False, palavra=None)}, fim_de_palavra=False, palavra=No
ne), 'y': NoTrie(filhos={'n': NoTrie(filhos={'c': NoTrie(filhos={}, fim_de_palavra=True, palavra='async')}, fim_de_palavra=False, palavra=None)}, fim_de_palavra=False,
palavra=None)}, fim_de_palavra=True, palavra='as'), 'w': NoTrie(filhos={'a': NoTrie(filhos={'i': NoTrie(filhos={'t': NoTrie(filhos={}, fim_de_palavra=True, palavra='aw
ait')}, fim_de_palavra=False, palavra=None)}, fim_de_palavra=False, palavra=None)}, fim_de_palavra=False, palavra=None)}, fim_de_palavra=False, palavra=None), 'b': NoT
rie(filhos={'r': NoTrie(filhos={'e': NoTrie(filhos={'a': NoTrie(filhos={'k': NoTrie(filhos={}, fim_de_palavra=True, palavra='break')}, fim_de_palavra=False, palavra=No
ne)}, fim_de_palavra=False, palavra=None)}, fim_de_palavra=False, palavra=None)}, fim_de_palavra=False, palavra=None), 'c': NoTrie(filhos={'l': NoTrie(filhos={'a': NoT
rie(filhos={'s': NoTrie(filhos={'s': NoTrie(filhos={}, fim_de_palavra=True, palavra='class')}, fim_de_palavra=False, palavra=None)}, fim_de_palavra=False, palavra=Non
e)}, fim_de_palavra=False, palavra=None), 'o': NoTrie(filhos={'n': NoTrie(filhos={'t': NoTrie(filhos={'i': NoTrie(filhos={'n': NoTrie(filhos={'u': NoTrie(filhos={'e':
NoTrie(filhos={}, fim_de_palavra=True, palavra='continue')}, fim_de_palavra=False, palavra=None)}, fim_de_palavra=False, palavra=None)}, fim_de_palavra=False, palavra=
None)}, fim_de_palavra=False, palavra=None)}, fim_de_palavra=False, palavra=None)}, fim_de_palavra=False, palavra=None)}, fim_de_palavra=False, palavra=None), 'd': NoT
rie(filhos={'e': NoTrie(filhos={'f': NoTrie(filhos={}, fim_de_palavra=True, palavra='def'), 'l': NoTrie(filhos={}, fim_de_palavra=True, palavra='del')}, fim_de_palavra
=False, palavra=None)}, fim_de_palavra=False, palavra=None), 'e': NoTrie(filhos={'l': NoTrie(filhos={'i': NoTrie(filhos={'f': NoTrie(filhos={}, fim_de_palavra=True, pa
lavra='elif')}, fim_de_palavra=False, palavra=None), 's': NoTrie(filhos={'e': NoTrie(filhos={}, fim_de_palavra=True, palavra='else')}, fim_de_palavra=False, palavra=No
ne)}, fim_de_palavra=False, palavra=None), 'x': NoTrie(filhos={'c': NoTrie(filhos={'e': NoTrie(filhos={'p': NoTrie(filhos={'t': NoTrie(filhos={}, fim_de_palavra=True,
palavra='except')}, fim_de_palavra=False, palavra=None)}, fim_de_palavra=False, palavra=None)}, fim_de_palavra=False, palavra=None)}, fim_de_palavra=False, palavra=Non
e)}, fim_de_palavra=False, palavra=None), 'f': NoTrie(filhos={'i': NoTrie(filhos={'n': NoTrie(filhos={'a': NoTrie(filhos={'l': NoTrie(filhos={'l': NoTrie(filhos={'y':
NoTrie(filhos={}, fim_de_palavra=True, palavra='finally')}, fim_de_palavra=False, palavra=None)}, fim_de_palavra=False, palavra=None)}, fim_de_palavra=False, palavra=N
one)}, fim_de_palavra=False, palavra=None)}, fim_de_palavra=False, palavra=None), 'o': NoTrie(filhos={'r': NoTrie(filhos={}, fim_de_palavra=True, palavra='for')}, fim_
de_palavra=False, palavra=None), 'r': NoTrie(filhos={'o': NoTrie(filhos={'m': NoTrie(filhos={}, fim_de_palavra=True, palavra='from')}, fim_de_palavra=False, palavra=No
ne)}, fim_de_palavra=False, palavra=None)}, fim_de_palavra=False, palavra=None), 'g': NoTrie(filhos={'l': NoTrie(filhos={'o': NoTrie(filhos={'b': NoTrie(filhos={'a': N
oTrie(filhos={'l': NoTrie(filhos={}, fim_de_palavra=True, palavra='global')}, fim_de_palavra=False, palavra=None)}, fim_de_palavra=False, palavra=None)}, fim_de_palavr
a=False, palavra=None)}, fim_de_palavra=False, palavra=None)}, fim_de_palavra=False, palavra=None), 'i': NoTrie(filhos={'f': NoTrie(filhos={}, fim_de_palavra=True, pal
avra='if'), 'm': NoTrie(filhos={'p': NoTrie(filhos={'o': NoTrie(filhos={'r': NoTrie(filhos={'t': NoTrie(filhos={}, fim_de_palavra=True, palavra='import')}, fim_de_pala
vra=False, palavra=None)}, fim_de_palavra=False, palavra=None)}, fim_de_palavra=False, palavra=None)}, fim_de_palavra=False, palavra=None), 'n': NoTrie(filhos={}, fim_
de_palavra=True, palavra='in'), 's': NoTrie(filhos={}, fim_de_palavra=True, palavra='is')}, fim_de_palavra=False, palavra=None), 'l': NoTrie(filhos={'a': NoTrie(filhos
={'m': NoTrie(filhos={'b': NoTrie(filhos={'d': NoTrie(filhos={'a': NoTrie(filhos={}, fim_de_palavra=True, palavra='lambda')}, fim_de_palavra=False, palavra=None)}, fim
_de_palavra=False, palavra=None)}, fim_de_palavra=False, palavra=None)}, fim_de_palavra=False, palavra=None)}, fim_de_palavra=False, palavra=None), 'n': NoTrie(filhos=
{'o': NoTrie(filhos={'n': NoTrie(filhos={'l': NoTrie(filhos={'o': NoTrie(filhos={'c': NoTrie(filhos={'a': NoTrie(filhos={'l': NoTrie(filhos={}, fim_de_palavra=True, pa
lavra='nonlocal')}, fim_de_palavra=False, palavra=None)}, fim_de_palavra=False, palavra=None)}, fim_de_palavra=False, palavra=None)}, fim_de_palavra=False, palavra=Non
e)}, fim_de_palavra=False, palavra=None), 't': NoTrie(filhos={}, fim_de_palavra=True, palavra='not')}, fim_de_palavra=False, palavra=None)}, fim_de_palavra=False, pala
vra=None), 'o': NoTrie(filhos={'r': NoTrie(filhos={}, fim_de_palavra=True, palavra='or')}, fim_de_palavra=False, palavra=None), 'p': NoTrie(filhos={'a': NoTrie(filhos=
{'s': NoTrie(filhos={'s': NoTrie(filhos={}, fim_de_palavra=True, palavra='pass')}, fim_de_palavra=False, palavra=None)}, fim_de_palavra=False, palavra=None)}, fim_de_p
alavra=False, palavra=None), 'r': NoTrie(filhos={'a': NoTrie(filhos={'i': NoTrie(filhos={'s': NoTrie(filhos={'e': NoTrie(filhos={}, fim_de_palavra=True, palavra='rais
e')}, fim_de_palavra=False, palavra=None)}, fim_de_palavra=False, palavra=None)}, fim_de_palavra=False, palavra=None), 'e': NoTrie(filhos={'t': NoTrie(filhos={'u': NoT
rie(filhos={'r': NoTrie(filhos={'n': NoTrie(filhos={}, fim_de_palavra=True, palavra='return')}, fim_de_palavra=False, palavra=None)}, fim_de_palavra=False, palavra=Non
e)}, fim_de_palavra=False, palavra=None)}, fim_de_palavra=False, palavra=None)}, fim_de_palavra=False, palavra=None), 't': NoTrie(filhos={'r': NoTrie(filhos={'y': NoTr
ie(filhos={}, fim_de_palavra=True, palavra='try')}, fim_de_palavra=False, palavra=None)}, fim_de_palavra=False, palavra=None), 'w': NoTrie(filhos={'h': NoTrie(filhos=
{'i': NoTrie(filhos={'l': NoTrie(filhos={'e': NoTrie(filhos={}, fim_de_palavra=True, palavra='while')}, fim_de_palavra=False, palavra=None)}, fim_de_palavra=False, pal
avra=None)}, fim_de_palavra=False, palavra=None), 'i': NoTrie(filhos={'t': NoTrie(filhos={'h': NoTrie(filhos={}, fim_de_palavra=True, palavra='with')}, fim_de_palavra=
False, palavra=None)}, fim_de_palavra=False, palavra=None)}, fim_de_palavra=False, palavra=None), 'y': NoTrie(filhos={'i': NoTrie(filhos={'e': NoTrie(filhos={'l': NoTr
ie(filhos={'d': NoTrie(filhos={}, fim_de_palavra=True, palavra='yield')}, fim_de_palavra=False, palavra=None)}, fim_de_palavra=False, palavra=None)}, fim_de_palavra=Fa
lse, palavra=None)}, fim_de_palavra=False, palavra=None)}, fim_de_palavra=False, palavra=None)
```

In [6]:

```python
# Descrição: esta célula implementa a busca de uma palavra exata na Trie.
# A função percorre a árvore caractere por caractere e só retorna verdadeiro
# quando o último nó visitado também marca o fim de uma palavra do dicionário.
# Complexidade: O(m) no tempo, em que m é o tamanho da palavra buscada;
# O(1) no espaço auxiliar, desconsiderando a lista de caminho usada para visualização.

def buscar_palavra_exata(raiz, palavra):
    no_atual = raiz
    caminho = []

    for caractere in palavra:
        caminho.append(caractere)
        if caractere not in no_atual.filhos:
            return False, caminho
        no_atual = no_atual.filhos[caractere]

    return no_atual.fim_de_palavra, caminho
```

In [7]:

```python
# Descrição: esta célula implementa a busca por prefixo na Trie.
# A função verifica se existe um caminho completo para todos os caracteres
# digitados pelo usuário.
# Complexidade: O(p) no tempo, em que p é o tamanho do prefixo;
# O(1) no espaço auxiliar, desconsiderando a lista de caminho usada para visualização.

def buscar_prefixo(raiz, prefixo):
    no_atual = raiz
    caminho = []

    for caractere in prefixo:
        caminho.append(caractere)
        if caractere not in no_atual.filhos:
            return None, caminho
        no_atual = no_atual.filhos[caractere]

    return no_atual, caminho
```

In [8]:

```python
# Descrição: esta célula implementa o algoritmo de autocompletar.
# Primeiro a função localiza o nó correspondente ao prefixo digitado; em seguida,
# percorre em profundidade a subárvore desse nó para coletar todas as palavras
# completas que começam com o prefixo.
# Complexidade: O(p + s) no tempo, em que p é o tamanho do prefixo e s é o número
# de nós visitados na subárvore do prefixo; O(h + r) no espaço, em que h é a altura
# da subárvore percorrida pela recursão e r é a quantidade de sugestões retornadas.

def autocompletar(raiz, prefixo):
    no_prefixo, caminho_prefixo = buscar_prefixo(raiz, prefixo)

    if no_prefixo is None:
        return [], caminho_prefixo, []

    sugestoes = []
    visitados = []

    def coletar_palavras(no_atual, caminho_atual):
        if no_atual.fim_de_palavra:
            sugestoes.append(no_atual.palavra)
            visitados.append(caminho_atual)

        for caractere in sorted(no_atual.filhos):
            coletar_palavras(no_atual.filhos[caractere], caminho_atual + caractere)

    coletar_palavras(no_prefixo, prefixo)
    return sugestoes, caminho_prefixo, visitados
```

In [9]:

```python
# Descrição: esta célula implementa a exibição letra a letra solicitada no enunciado.
# Para cada novo caractere digitado, o programa calcula e mostra as possíveis
# palavras que começam com o prefixo parcial formado até aquele momento.
# Complexidade: O(k*p + S) no tempo para uma entrada de k caracteres, considerando
# as buscas de prefixo parciais e as subárvores percorridas em cada passo; O(r) no
# espaço por passo, em que r é a quantidade de sugestões retornadas naquele prefixo.

def mostrar_sugestoes_letra_a_letra(raiz, texto_digitado):
    texto_digitado = texto_digitado.strip()

    if not texto_digitado:
        print("Nenhum prefixo foi digitado.")
        return

    for indice in range(1, len(texto_digitado) + 1):
        prefixo_parcial = texto_digitado[:indice]
        sugestoes, _, _ = autocompletar(raiz, prefixo_parcial)
        print(f"Prefixo digitado: {prefixo_parcial}")
        print("Sugestões:")
        if sugestoes:
            for palavra in sugestoes:
                print(palavra)
        else:
            print("Nenhuma palavra encontrada para esse prefixo.")
        print()
```

In [10]:

```python
# Descrição: esta célula permite que o usuário digite o início de uma palavra.
# O programa exibe as sugestões letra a letra, mostrando a lista de palavras
# possíveis para cada prefixo parcial digitado, conforme o exemplo do enunciado.

prefixo_digitado = input("Digite o início de uma palavra reservada de Python: ")
mostrar_sugestoes_letra_a_letra(raiz_trie, prefixo_digitado)
```

```
Prefixo digitado: f
Sugestões:
finally
for
from

Prefixo digitado: fa
Sugestões:
Nenhuma palavra encontrada para esse prefixo.
```

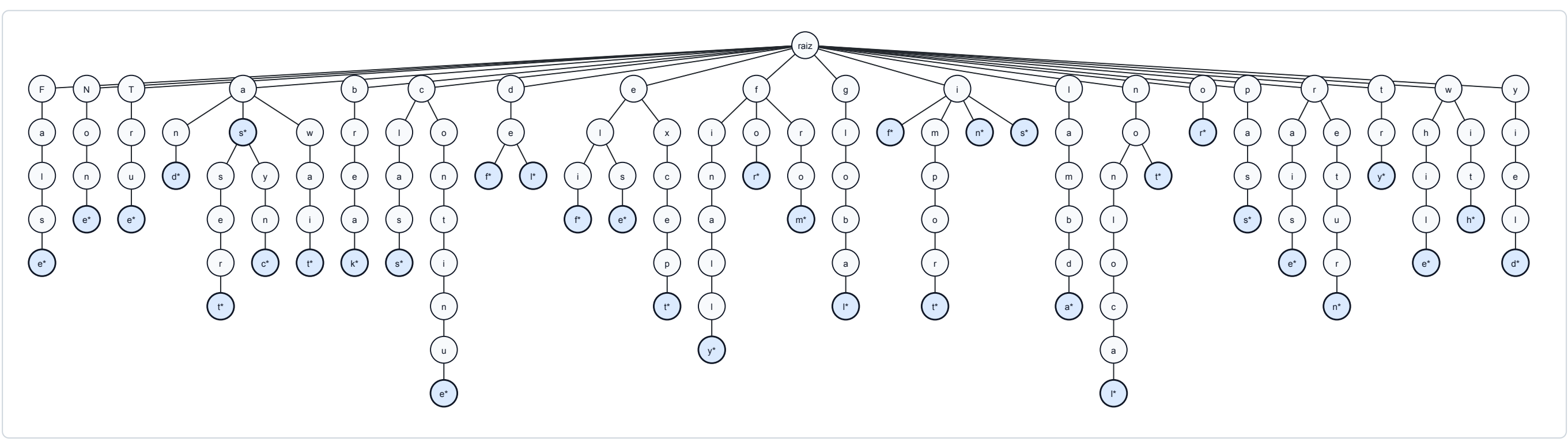
In [11]:

```python
# Descrição: esta célula contém a rotina para exibir a árvore montada.
# A exibição usa HTML/SVG, com nós em círculos e arestas em linhas.

def coletar_nos_e_arestas(raiz):
    nos = [("", raiz, "raiz", 0)]
    arestas = []

    def visitar(no_atual, caminho, profundidade):
        for caractere in sorted(no_atual.filhos):
            filho = no_atual.filhos[caractere]
            caminho_filho = caminho + caractere
            rotulo = caractere + ("*" if filho.fim_de_palavra else "")
            nos.append((caminho_filho, filho, rotulo, profundidade + 1))
            arestas.append((caminho, caminho_filho))
            visitar(filho, caminho_filho, profundidade + 1)

    visitar(raiz, "", 0)
    return nos, arestas


def calcular_posicoes_trie(raiz):
    nos, arestas = coletar_nos_e_arestas(raiz)
    folhas = []

    def visitar_folhas(no_atual, caminho):
        if not no_atual.filhos:
            folhas.append(caminho)
            return

        for caractere in sorted(no_atual.filhos):
            visitar_folhas(no_atual.filhos[caractere], caminho + caractere)

    visitar_folhas(raiz, "")
    indice_folha = {caminho: indice for indice, caminho in enumerate(folhas)}
    posicoes = {}

    def posicionar(no_atual, caminho, profundidade):
        if not no_atual.filhos:
            x = 70 + indice_folha[caminho] * 80
        else:
            filhos = []
            for caractere in sorted(no_atual.filhos):
                caminho_filho = caminho + caractere
                posicionar(no_atual.filhos[caractere], caminho_filho, profundidade + 1)
                filhos.append(posicoes[caminho_filho][0])
            x = sum(filhos) / len(filhos)

        y = 60 + profundidade * 78
        posicoes[caminho] = (x, y)

    posicionar(raiz, "", 0)
    return nos, arestas, posicoes


def exibir_arvore_montada(raiz, caminhos_destacados=None, no_atual=None, titulo="Árvore Trie montada"):
    caminhos_destacados = set(caminhos_destacados or [])
    nos, arestas, posicoes = calcular_posicoes_trie(raiz)
    largura = max(900, int(max(x for x, _ in posicoes.values()) + 90))
    altura = max(260, int(max(y for _, y in posicoes.values()) + 80))

    partes = [
        f'<h3 style="font-family:Arial,sans-serif">{html.escape(titulo)}</h3>',
        f'<svg width="100%" height="{altura}" viewBox="0 0 {largura} {altura}" '
        'style="background:#ffffff;border:1px solid #d0d7de;border-radius:6px">'
    ]

    for origem, destino in arestas:
        x1, y1 = posicoes[origem]
        x2, y2 = posicoes[destino]
        cor = "#16a34a" if origem in caminhos_destacados and destino in caminhos_destacados else "#24292f"
        largura_linha = 4 if cor == "#16a34a" else 2
        partes.append(
            f'<line x1="{x1}" y1="{y1}" x2="{x2}" y2="{y2}" '
            f'stroke="{cor}" stroke-width="{largura_linha}" />'
        )

    for caminho, no, rotulo, _ in nos:
        x, y = posicoes[caminho]
        if caminho == no_atual:
            cor = "#f59e0b"
            largura_borda = 5
        elif caminho in caminhos_destacados:
            cor = "#22c55e"
            largura_borda = 4
        elif no.fim_de_palavra:
            cor = "#dbeafe"
            largura_borda = 3
        else:
            cor = "#f8fafc"
            largura_borda = 2

        partes.append(
            f'<circle cx="{x}" cy="{y}" r="24" fill="{cor}" '
            f'stroke="#111827" stroke-width="{largura_borda}" />'
        )
        partes.append(
            f'<text x="{x}" y="{y + 7}" text-anchor="middle" '
            'font-size="16" font-family="Arial, sans-serif" fill="#111827">'
            f'{html.escape(rotulo)}</text>'
        )

    partes.append("</svg>")
    partes.append(
        '<div style="font-family:Arial,sans-serif;margin-top:8px">'
        '<strong>Legenda:</strong> raiz = início da Trie; * = nó que encerra uma palavra; '
        '<span style="color:#22c55e;font-weight:700">verde</span> = caminho visitado; '
        '<span style="color:#f59e0b;font-weight:700">laranja</span> = nó atual.'
        '</div>'
    )
    display(HTML("".join(partes)))
```

In [12]:

```python
# Descrição: esta célula exibe a árvore completa montada com o dicionário de
# palavras reservadas de Python.

exibir_arvore_montada(raiz_trie)
```

Árvore Trie montada


raiz
F N T a b c d e f g i l n o p r t w y
a o r n s* w r l o e l x i o r l f* m n* s* a o r* a a e r h i i
l n u d* s y a e a n f* l* i s c n r* o o p m n t* s i t y* i t e
s e* e* e n i a s t f* e* e a m* b o b l s* s u l h* l
e* r c* t* k* s* i p l a r d o e* r e* d*
t* n t* l l* t* a* c n*
u y* a
e* l*

Legenda: raiz = início da Trie; * = nó que encerra uma palavra; **verde** = caminho visitado; **laranja** = nó atual.

In [13]:

```python
# Descrição: esta célula define funções auxiliares para mostrar visualmente,
# de forma didática, a execução das buscas. Este código de visualização fica
# separado das células que implementam os algoritmos de busca.

def caminhos_parciais(caminho):
    caminhos = [""]
    acumulado = ""
    for caractere in caminho:
        acumulado += caractere
        caminhos.append(acumulado)
    return caminhos


def mostrar_passos_busca(titulo, palavra_ou_prefixo, caminho, encontrado):
    blocos = []
    acumulado = ""

    for indice, caractere in enumerate(caminho, start=1):
        acumulado += caractere
        caminhos = caminhos_parciais(acumulado)
        status = "encontrado" if indice < len(caminho) or encontrado else "não encontrado"
        blocos.append(
            f'<div style="font-family:Arial,sans-serif;font-weight:700;margin:18px 0 6px">'
            f'Passo {indice}: lê {html.escape(caractere)} | caminho {html.escape(acumulado)} | {status}'
            '</div>'
        )
        nos, arestas, posicoes = calcular_posicoes_trie(raiz_trie)
        largura = max(900, int(max(x for x, _ in posicoes.values()) + 90))
        altura = max(260, int(max(y for _, y in posicoes.values()) + 80))
        partes_svg = [
            f'<svg width="100%" height="{altura}" viewBox="0 0 {largura} {altura}" '
            'style="background:#ffffff;border:1px solid #d0d7de;border-radius:6px">'
        ]
        for origem, destino in arestas:
            x1, y1 = posicoes[origem]
            x2, y2 = posicoes[destino]
            cor = "#16a34a" if origem in caminhos and destino in caminhos else "#24292f"
            partes_svg.append(
                f'<line x1="{x1}" y1="{y1}" x2="{x2}" y2="{y2}" '
                f'stroke="{cor}" stroke-width="{4 if cor == "#16a34a" else 2}" />'
            )
        for caminho_no, no, rotulo, _ in nos:
            x, y = posicoes[caminho_no]
            cor = "#f59e0b" if caminho_no == acumulado else "#22c55e" if caminho_no in caminhos else "#dbeafe" if no.fim_de_palavra else "#f8fafc"
            partes_svg.append(
                f'<circle cx="{x}" cy="{y}" r="24" fill="{cor}" stroke="#111827" stroke-width="3" />'
            )
            partes_svg.append(
                f'<text x="{x}" y="{y + 7}" text-anchor="middle" font-size="16" '
                f'font-family="Arial, sans-serif" fill="#111827">{html.escape(rotulo)}</text>'
            )
        partes_svg.append("</svg>")
        blocos.append("".join(partes_svg))

    resultado = "encontrado" if encontrado else "não encontrado"
    display(HTML(
        f'<h3 style="font-family:Arial,sans-serif">{html.escape(titulo)}</h3>'
        f'<div style="font-family:Arial,sans-serif;margin-bottom:8px">Entrada: '
        f'<strong>{html.escape(palavra_ou_prefixo)}</strong> | Resultado: <strong>{resultado}</strong></div>'
        + "".join(blocos)
    ))


def mostrar_autocompletar_visual(prefixo):
    sugestoes, caminho_prefixo, visitados = autocompletar(raiz_trie, prefixo)
    caminhos = set(caminhos_parciais(prefixo))
    for palavra in visitados:
        caminhos.update(caminhos_parciais(palavra))

    titulo = f"Autocompletar para o prefixo '{prefixo}'"
    exibir_arvore_montada(raiz_trie, caminhos_destacados=caminhos, no_atual=prefixo, titulo=titulo)
    print("Sugestões:")
    if sugestoes:
        for palavra in sugestoes:
            print(palavra)
    else:
        print("Nenhuma palavra encontrada para esse prefixo.")
```

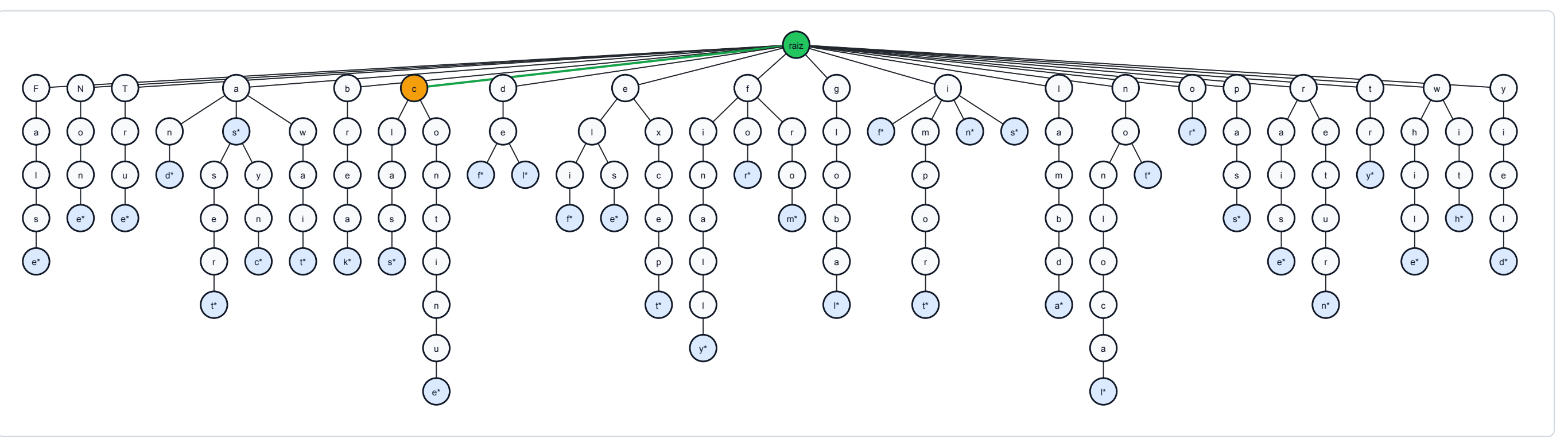
In [14]:

```python
# Descrição: esta célula mostra visualmente a execução da busca por palavra exata.
# O exemplo verifica se a palavra 'class' existe no dicionário da Trie.

palavra_teste = "class"
encontrou_palavra, caminho_palavra = buscar_palavra_exata(raiz_trie, palavra_teste)
mostrar_passos_busca("Execução visual da busca por palavra exata", palavra_teste, caminho_palavra, encontrou_palavra)
```

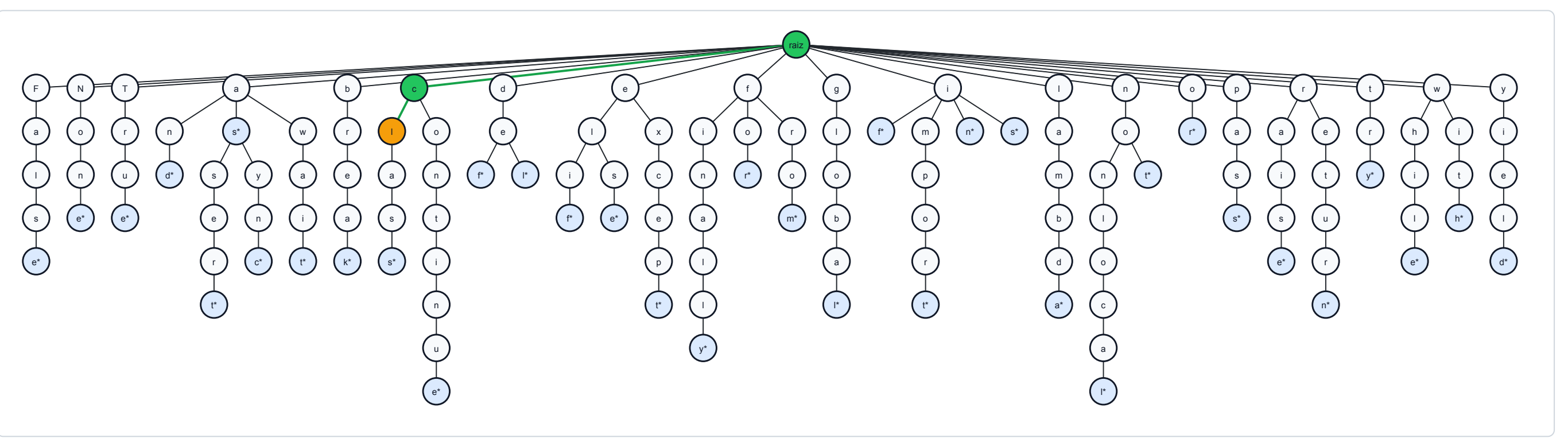
Execução visual da busca por palavra exata

Entrada: **class** | Resultado: **encontrado**

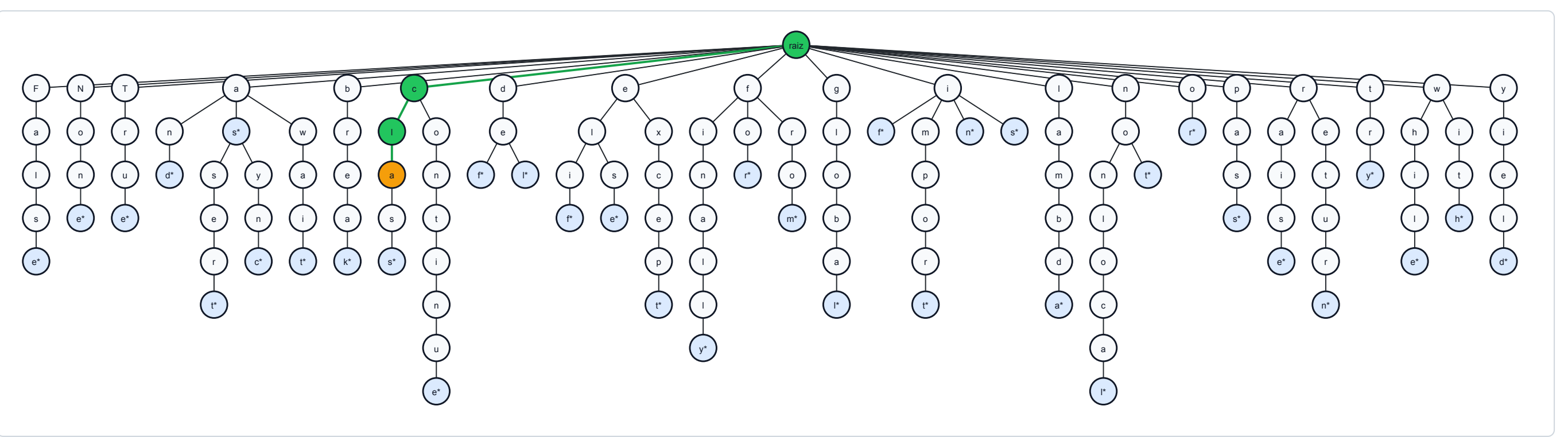
Passo 1: lê c | caminho c | encontrado


raiz
F N T a b c d e f g i l n o p r t w y
a o r n s* w r l o e l x i o r l f* m n* s* a o r* a a e r h i i
l n u d* s y a e a n f* l* i s c n r* o o p m n t* s i t y* i t e
s e* e* e n i a s t f* e* e a m* b o b l s* s u l h* l
e* r c* t* k* s* i p l a r d o e* r e* d*
t* n t* l l* t* a* c n*
u y* a
e* l*

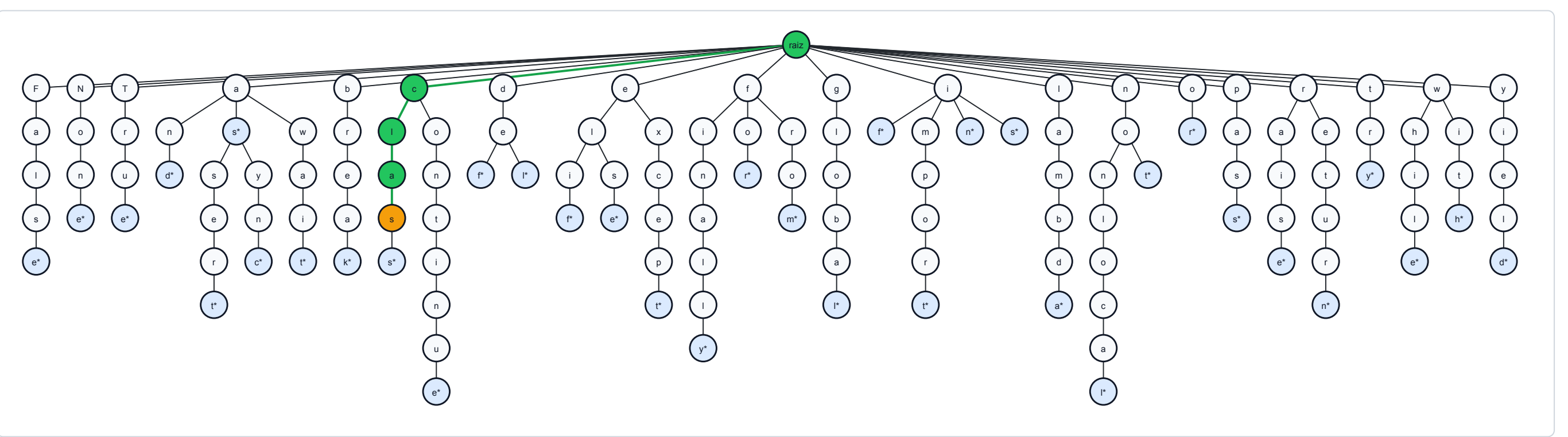
Passo 2: lê l | caminho cl | encontrado


raiz
F N T a b c d e f g i l n o p r t w y
a o r n s* w r l o e l x i o r l f* m n* s* a o r* a a e r h i i
l n u d* s y a e a n f* l* i s c n r* o o p m n t* s i t y* i t e
s e* e* e n i a s t f* e* e a m* b o b l s* s u l h* l
e* r c* t* k* s* i p l a r d o e* r e* d*
t* n t* l l* t* a* c n*
u y* a
e* l*

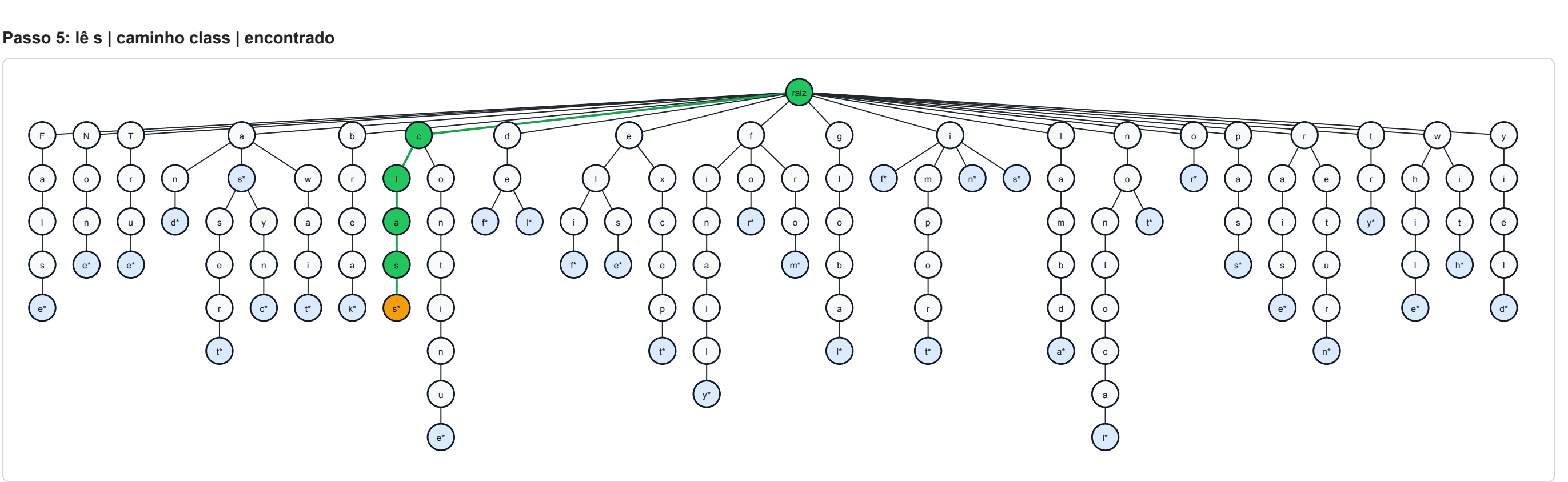
Passo 3: lê a | caminho cla | encontrado


raiz
F N T a b c d e f g i l n o p r t w y
a o r n s* w r l o e l x i o r l f* m n* s* a o r* a a e r h i i
l n u d* s y a e a n f* l* i s c n r* o o p m n t* s i t y* i t e
s e* e* e n i a s t f* e* e a m* b o b l s* s u l h* l
e* r c* t* k* s* i p l a r d o e* r e* d*
t* n t* l l* t* a* c n*
u y* a
e* l*

Passo 4: lê s | caminho clas | encontrado


raiz
F N T a b c d e f g i l n o p r t w y
a o r n s* w r l o e l x i o r l f* m n* s* a o r* a a e r h i i
l n u d* s y a e a n f* l* i s c n r* o o p m n t* s i t y* i t e
s e* e* e n i a s t f* e* e a m* b o b l s* s u l h* l
e* r c* t* k* s* i p l a r d o e* r e* d*
t* n t* l l* t* a* c n*
u y* a
e* l*

Passo 5: lê s | caminho class | encontrado


raiz
F N T a b c d e f g i l n o p r t w y
a o r n s* w r l o e l x i o r l f* m n* s* a o r* a a e r h i i
l n u d* s y a e a n f* l* i s c n r* o o p m n t* s i t y* i t e
s e* e* e n i a s t f* e* e a m* b o b l s* s u l h* l
e* r c* t* k* s* i p l a r d o e* r e* d*
t* n t* l l* t* a* c n*
u y* a
e* l*

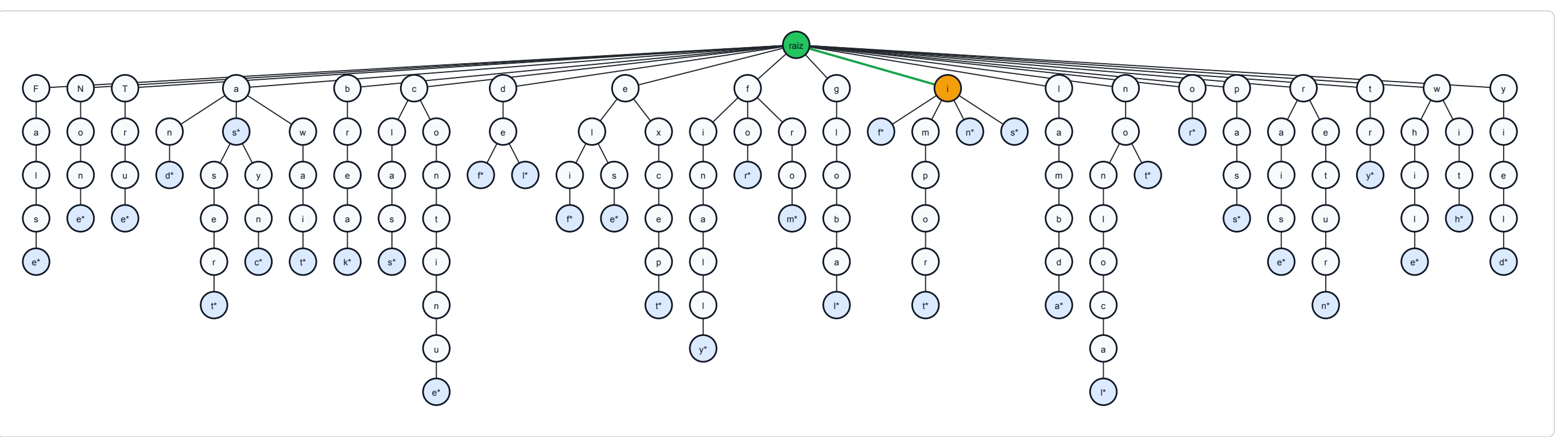
In [15]:

```python
# Descrição: esta célula mostra visualmente a execução da busca por prefixo.
# O exemplo verifica se o prefixo 'im' existe na Trie.

prefixo_teste = "im"
no_prefixo, caminho_prefixo = buscar_prefixo(raiz_trie, prefixo_teste)
mostrar_passos_busca("Execução visual da busca por prefixo", prefixo_teste, caminho_prefixo, no_prefixo is not None)
```

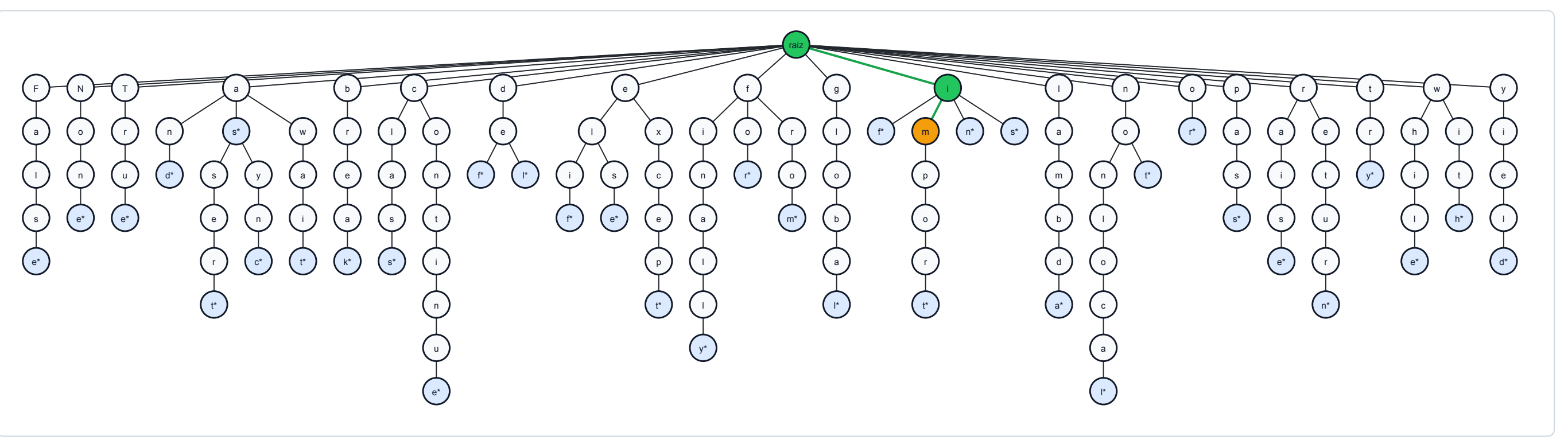
Execução visual da busca por prefixo

Entrada: **im** | Resultado: **encontrado**

Passo 1: lê i | caminho i | encontrado


raiz
F N T a b c d e f g i l n o p r t w y
a o r n s* w r l o e l x i o r l f* m n* s* a o r* a a e r h i i
l n u d* s y a e a n f* l* i s c n r* o o p m n t* s i t y* i t e
s e* e* e n i a s t f* e* e a m* b o b l s* s u l h* l
e* r c* t* k* s* i p l a r d o e* r e* d*
t* n t* l l* t* a* c n*
u y* a
e* l*

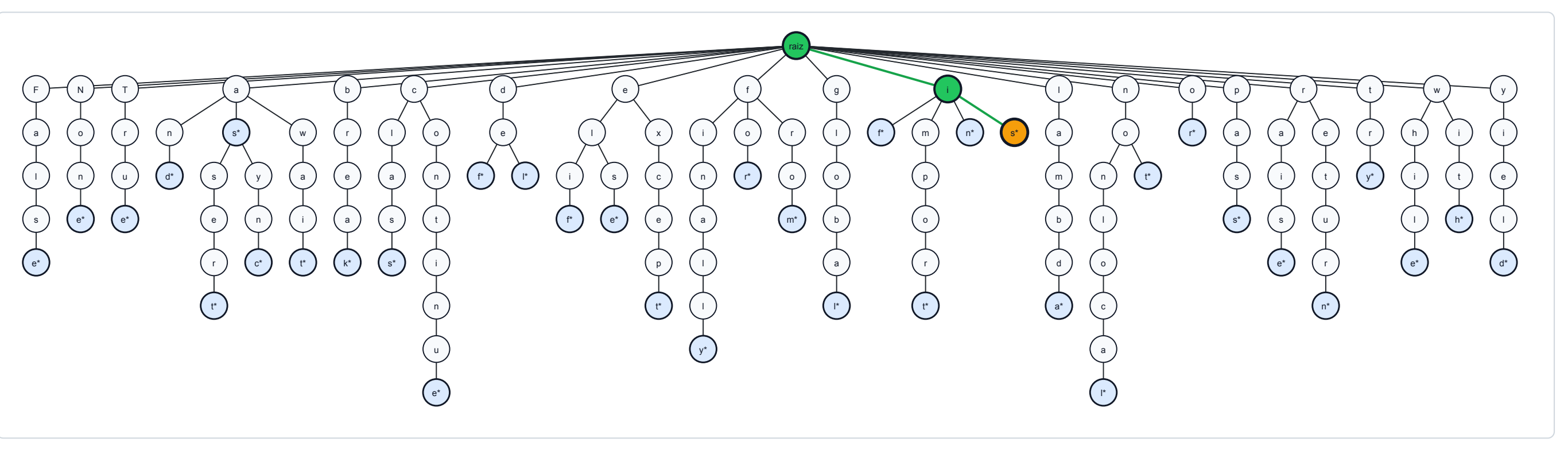
Passo 2: lê m | caminho im | encontrado


raiz
F N T a b c d e f g i l n o p r t w y
a o r n s* w r l o e l x i o r l f* m n* s* a o r* a a e r h i i
l n u d* s y a e a n f* l* i s c n r* o o p m n t* s i t y* i t e
s e* e* e n i a s t f* e* e a m* b o b l s* s u l h* l
e* r c* t* k* s* i p l a r d o e* r e* d*
t* n t* l l* t* a* c n*
u y* a
e* l*

In [16]:

```python
# Descrição: esta célula mostra visualmente a execução do autocompletar.
# O exemplo destaca a subárvore percorrida para sugerir palavras que começam com 'is'.

mostrar_autocompletar_visual("is")
```

Autocompletar para o prefixo 'is'


raiz
F N T a b c d e f g i l n o p r t w y
a o r n s* w r l o e l x i o r l f* m n* s* a o r* a a e r h i i
l n u d* s y a e a n f* l* i s c n r* o o p m n t* s i t y* i t e
s e* e* e n i a s t f* e* e a m* b o b l s* s u l h* l
e* r c* t* k* s* i p l a r d o e* r e* d*
t* n t* l l* t* a* c n*
u y* a
e* l*

Legenda: raiz = início da Trie; * = nó que encerra uma palavra; **verde** = caminho visitado; **laranja** = nó atual.

```
Sugestões:
is
```

In [17]:

```python
# Descrição: esta célula executa testes simples para validar as operações principais
# da Trie com o dicionário de palavras reservadas de Python.

assert buscar_palavra_exata(raiz_trie, "class")[0] is True
assert buscar_palavra_exata(raiz_trie, "classe")[0] is False
assert buscar_prefixo(raiz_trie, "im")[0] is not None
assert buscar_prefixo(raiz_trie, "xyz")[0] is None
assert autocompletar(raiz_trie, "is")[0] == ["is"]
assert autocompletar(raiz_trie, "a")[0] == ["and", "as", "assert", "async", "await"]

print("Todos os testes passaram.")
```

```
Todos os testes passaram.
```

Resultados pedidos no enunciado

O programa montou uma Árvore Trie usando como dicionário as palavras reservadas da linguagem Python. Após a montagem, o usuário pode digitar o início de uma palavra e o algoritmo retorna, em tela, todas as palavras do dicionário que começam com esse prefixo.

Exemplos de resultados esperados:

- Prefixo `a`: `and`, `as`, `assert`, `async`, `await`
- Prefixo `f`: `False`, `finally`, `for`, `from`
- Prefixo `is`: `is`
- Prefixo `no`: `nonlocal`, `not`
- Prefixo inexistente, como `xyz`: nenhuma palavra encontrada

Assim, a saída final atende ao enunciado: uma lista de palavras do dicionário que iniciam pelas letras digitadas pelo usuário.